Lab 4: VibeCoding - Full Chat History

Project: Anvil Dodge Repair Lab (63_anvil_dodge) | Assistant: Claude (Anthropic) | Date: 11 Oct 2026

ChatGPT chat link submitted by the student: https://chatgpt.com/share/6acb7534-3a34-83e8-8c3c-201fd49492bb

This is the complete Claude conversation, in order. Blue boxes are the student's messages (pasted files included in full). Orange boxes are Claude's replies, word for word. Grey italic lines describe the tool work Claude did between messages (running commands, tests). The final code is in the appendix.

Turn 1 - Uploads

STUDENT
[Uploaded files: 1791717985096_se_before.mp4, 1791717997772_se_after.mp4, 1791718093540_Lab_4_VibeCoding_Student_handout.pdf]

give files which are need to be pushed ready files

[Handout text, as extracted from the PDF:]

Lab 4: VibeCoding
Objective
Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account.
THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!
Duration: 60 minutes
Software Requirements: Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)
Instructions:
1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10. Have Separate Commits for each task
11. Do not raise PR to the main repo (SETAPESU26)
Deliverables: Upload the following to your Lab repo under Lab-4 folder,
a. Videos (before and after) b. Updated Code c. Chat history exported as a doc/pdf

[Claude's work between messages] Listed the uploads and read video metadata with ffprobe (before: 1864x1034, 11.5 s; after: 876x666, 41.6 s). Extracted frames from both videos. The before frame shows VS Code with project folder 63_anvil_dodge (main.py, game/, README.md) and the pygame window 'Anvil Dodge - Pygame Edition' with gray anvils. The after frames show a Game Over screen with 'Press [R] to Play Again', gold/red/gray anvils and dust particles.

CLAUDE
I can't produce the code or the chat-history file yet, because the repo's code and ReadMe weren't in what you uploaded. I only have the two videos and the lab handout.

From the videos, the repo is `63_anvil_dodge` (a pygame game with `main.py` and a `game/game_engine.py`). Your "after" video already shows the finished features:
- a Game Over screen with a Play Again prompt (R)
- anvils in different colours (gray, gold, red)
- landing particles
- a dimmed overlay when you are crushed

I can't see which bugs were broken at the start or what the ReadMe asks for. If I wrote the code from the video alone, it might not match what your instructor expects, and it wouldn't be your own work.

Please send:
1. The **README.md** from the repo.
2. The **original (broken) code**, meaning `main.py` and everything in the `game/` folder. Paste it or upload it.

Once I have those, I'll prepare a ready-to-push `Lab-4/` folder:
- `Lab-4/code/`: the fixed game with all the ReadMe features.
- `Lab-4/videos/`: your before and after videos.
- A commit script with one commit per task, as the handout requires.
- A chat-history PDF covering the fix and each feature prompt. This will be a real record of what we do.

Your "after" video is 41 seconds and the handout asks for about 10 seconds. I can trim it to 10 seconds if you want.

Turn 2 - main.py and ReadMe

STUDENT
[Pasted main.py:]

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Anvil Dodge - Pygame Edition")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

STUDENT
[Pasted the ReadMe:]

Anvil Dodge Repair Lab
This project is a modular survival dodger game using Pygame. It introduces students to falling hazard mechanics, boundary management, collision detection, and survival time tracking within a clean, object-oriented codebase.
What's Provided
A working Anvil Dodge game with:

* A player character that can move left and right across the ground line using keyboard inputs
* Heavy anvils spawned continuously from random horizontal positions falling toward the ground
* Collision detection when an anvil hits the player, triggering game over
* Real-time survival time tracking and a Game Over overlay with restart functionality

It has one deliberate bug and three optional features left as tasks to implement. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting.
Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started
Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies: pip install pygame
3. Run the game: python main.py

Controls: Left / A to move left, Right / D to move right, R to reset after Game Over.
Tasks to Complete
Each task must be completed using an iterative process involving LLM suggestions and your critical code review.
Task 1: Fix the player off-screen boundary bug
The player character can walk past the left and right screen edges into hidden space where falling anvils cannot hit them, exploiting an infinite survival time. Constrain the player so movement is strictly clamped within the visible boundaries of the display window.
Task 2: Implement dynamic difficulty scaling
The player character can walk past the left and right screen edges into hidden space where falling anvils cannot hit them, exploiting an infinite survival time. Constrain the player so movement is strictly clamped within the visible boundaries of the display window.
Task 3: Implement speed-based anvil warning tints
All falling hazards currently share the same grey appearance regardless of velocity. Apply visual hazard tiers to falling anvils by tinting higher-velocity anvils with distinct hot warning colors to signal urgent threats to the player.
Task 4: Implement ground impact FX & screen shake
Anvils reaching the floor vanish silently without physical presence. Add kinetic feedback on impact - such as localized dust cloud particles or a subtle camera shake whenever an anvil smashes into the ground line.
Expected Behavior

* The player cannot move beyond the visible left and right edges of the screen
* Anvils fall continuously from randomized X coordinates and clean up after passing below the screen
* Touching any falling anvil immediately triggers the Game Over screen and stops time tracking
* Pressing R after losing resets the player, clears all falling anvils, and restarts the survival timer

Folder Structure
anvil_dodge/
game/anvil.py, game/game_engine.py, game/player.py
main.py
README.md

Submission Checklist
Submission is only the following three things:
[] A 10-second video of gameplay before your changes, showing the bug/broken behavior
[] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
[] The Chat/LLM used page link, with the complete chat history

CLAUDE
You've sent `main.py` and the ReadMe, but not the three files in `game/` (`anvil.py`, `game_engine.py`, `player.py`). The ReadMe and the `main.py` interface are enough to rebuild them, so I'll write them against that spec and test them. If your originals differ, send them and I'll port the changes.

Turn 3 - Waiting

STUDENT
wait i will send those also

CLAUDE
Sure, go ahead and send `anvil.py`, `game_engine.py`, and `player.py`. I'll hold off until they arrive, so the fixes and features are built on your actual code.

Two things to note from the ReadMe:
- **Task 2 is a copy of Task 1.** The ReadMe's Task 2 text ("dynamic difficulty scaling") repeats the boundary-bug description. I'll treat Task 2 as difficulty scaling, meaning anvils spawn faster and fall faster over time. The title says that, and your "after" video seems to show it. Let me know if your instructor meant something different.
- **The submission is a chat link.** The ReadMe asks for the Chat/LLM page link with the full history. I'll still produce the PDF, but you may also need to share the link from this conversation.

Turn 4 - Original game files

STUDENT

[Pasted anvil.py:]

```
import random
import pygame


class Anvil:
    def __init__(self, screen_width):
        self.screen_width = screen_width
        self.width = 40
        self.height = 32
        self.x = random.randint(20, screen_width - self.width - 20)
        self.y = -self.height
        self.speed = random.uniform(4.5, 7.0)

    def update(self):
        self.y += self.speed

    def is_off_screen(self, screen_height):
        return self.y > screen_height + 10

    @property
    def rect(self):
        return pygame.Rect(int(self.x), int(self.y), self.width, self.height)

    def render(self, surface):
        top_rect = pygame.Rect(int(self.x) + 4, int(self.y), self.width - 8, 14)
        pygame.draw.rect(surface, (120, 120, 130), top_rect, border_radius=2)

        base_rect = pygame.Rect(int(self.x), int(self.y) + 14, self.width, 18)
        pygame.draw.rect(surface, (80, 80, 90), base_rect, border_radius=3)
        pygame.draw.rect(surface, (200, 200, 210), base_rect, width=1, border_radius=3)
```

STUDENT

[Pasted game_engine.py:]

```
import pygame
from game.player import Player
from game.anvil import Anvil


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.player = Player(width, height)
        self.anvils = []

        self.spawn_delay = 700
        self.last_spawn_time = pygame.time.get_ticks()

        self.start_ticks = pygame.time.get_ticks()
        self.survival_time = 0
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 52)
        self.font_medium = pygame.font.SysFont(None, 34)
        self.font_small = pygame.font.SysFont(None, 24)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move_left()
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move_right()

        self.player.update()

        self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000
```

```
            now = pygame.time.get_ticks()
            if now - self.last_spawn_time >= self.spawn_delay:
                self.anvils.append(Anvil(self.width))
                self.last_spawn_time = now

            player_rect = self.player.rect
            for anvil in self.anvils[:]:
                anvil.update()

                if player_rect.colliderect(anvil.rect):
                    self.game_state = "GAME_OVER"

                if anvil.is_off_screen(self.height):
                    self.anvils.remove(anvil)

        def reset(self):
            self.player = Player(self.width, self.height)
            self.anvils.clear()
            self.start_ticks = pygame.time.get_ticks()
            self.last_spawn_time = pygame.time.get_ticks()
            self.survival_time = 0
            self.game_state = "PLAYING"

        def render(self, screen):
            screen.fill((35, 38, 45))

            ground_y = self.height - 20
            pygame.draw.rect(screen, (70, 75, 85), (0, ground_y, self.width, 20))
            pygame.draw.line(screen, (160, 90, 40), (0, ground_y), (self.width, ground_y), 3)

            self.player.render(screen)
            for anvil in self.anvils:
                anvil.render(screen)

            time_surf = self.font_medium.render(f"Survival Time: {self.survival_time}s", True, (240, 240, 240))
            screen.blit(time_surf, (20, 20))

            inst_surf = self.font_small.render("Use [A/D] or [Arrow Keys] to Dodge", True, (170, 175, 185))
            screen.blit(inst_surf, (self.width - inst_surf.get_width() - 20, 25))

            if self.game_state == "GAME_OVER":
                overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
                overlay.fill((0, 0, 0, 190))
                screen.blit(overlay, (0, 0))

                over_surf = self.font_big.render("CRUSHED! GAME OVER", True, (235, 65, 65))
                screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 60))

                score_surf = self.font_medium.render(f"You survived: {self.survival_time} seconds", True, (255, 255, 255))
                screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, self.height // 2))

                restart_surf = self.font_small.render("Press [R] to Play Again", True, (200, 200, 200))
                screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))
```

STUDENT

[Pasted player.py:]

```
import pygame


class Player:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.width = 44
        self.height = 54
        self.speed = 7

        self.x = (screen_width // 2) - (self.width // 2)
        self.y = screen_height - self.height - 20

    def move_left(self):
        self.x -= self.speed

    def move_right(self):
        self.x += self.speed

    def update(self):
        pass

    @property
    def rect(self):
```

```
            return pygame.Rect(int(self.x), int(self.y), self.width, self.height)

        def render(self, surface):
            torso_rect = pygame.Rect(int(self.x) + 6, int(self.y) + 18, 32, 36)
            pygame.draw.rect(surface, (50, 130, 220), torso_rect, border_radius=4)

            pygame.draw.circle(surface, (230, 200, 160), (int(self.x) + 22, int(self.y) + 12), 12)
            helmet_rect = pygame.Rect(int(self.x) + 8, int(self.y) - 2, 28, 12)
            pygame.draw.rect(surface, (180, 180, 190), helmet_rect, border_radius=4)
```

[Claude's work between messages] Root cause of Task 1: Player.move_left/move_right change x with no limit and Player.update() is empty, so x can go negative or beyond the screen width where no anvil can reach.

[Claude's work between messages] Wrote the original code as a baseline (step0), then four snapshots: step1 = boundary clamp in Player.update(); step2 = difficulty scaling (spawn delay 700ms -> 250ms, speed bonus up to +6, Level counter); step3 = speed tint tiers in Anvil.render(); step4 = DustParticle class (new game/particles.py), ground-impact detection, screen shake via an off-screen canvas. All compile.

[Claude's work between messages] pygame was not installable in the sandbox (no network), so the logic was tested against a minimal stand-in pygame module. First test run failed because my test script loaded the step2 module where it needed step3 (a bug in the test, not the game); fixed the script and re-ran:

```
step0 player.x after walking left: -1772 (bug: off-screen)
Task 1 OK: clamped to [0, 656]
Task 2 OK: t=0 -> (700, 0.0) | t=40s -> delay 250ms, speed bonus 4.0, level 5
   caps hold at 250ms / +6.0 speed
   reset restores base difficulty
Task 3 OK: tiers grey <6.5 < gold <9.0 <= red
Task 4: particles max=24, shake triggered=True, over=True, anvils on screen=2
Game over / time freeze / R-reset OK
ALL TESTS PASSED
```

[Claude's work between messages] Built /Lab-4 (code, videos renamed before.mp4/after.mp4) and git_commit_steps/ (bash + PowerShell scripts, one commit per task). Found and fixed a bug in my commit script (step 0 would have staged the videos too), then dry-ran it in a scratch git repo: 7 commits, one per task, final code identical to the delivered code. Generated an A4 summary PDF of the chat.

CLAUDE
I have all four files now. First I'll read the PDF skill for the chat-history file, then build the fixes.

[...tool work above...]

I've made the `Lab-4/` folder, with your original `main.py` unchanged. I couldn't run the game itself: pygame wouldn't install in my environment, so I tested the game logic against a stand-in pygame, and all of those tests passed. The dust, shake and colours still need a look on your machine, so run `python main.py` once before you push.

What each task does:
- **Task 1 (boundary bug):** the player's `update()` was empty, so nothing limited `x`. I added a clamp so the player stays between 0 and `screen_width - player_width`.
- **Task 2 (difficulty):** anvils spawn faster (700 ms down to 250 ms) and fall faster (up to +6 speed) the longer you survive. A small "Level N" appears under the timer, and pressing R resets all of it.
- **Task 3 (tints):** slow anvils stay gray, fast ones turn gold, and very fast ones turn red.
- **Task 4 (impact FX):** when an anvil hits the ground, 14 dust particles puff up and the screen shakes briefly. Faster anvils shake it harder.

Your files (outputs folder):
- `Lab-4/anvil_dodge/`: final code (`main.py`, `game/`, including a new `game/particles.py`).
- `Lab-4/videos/before.mp4` and `after.mp4`: your two uploaded videos, renamed.
- `Lab-4/chat_history.pdf`: a condensed record of this session.
- `git_commit_steps/`: scripts that make one commit per task.

To push: copy `Lab-4/` and `git_commit_steps/` into the root of your personal repo. Run `bash git_commit_steps/commit_tasks.sh` (Git Bash), or the `.ps1` file in PowerShell, then `git push`. I dry-ran it in a scratch repo and it produces seven commits, one per task, and the final code matches `Lab-4/anvil_dodge`. Delete `git_commit_steps/` before pushing if you don't want it in the repo.

Three things to check:
- **Videos:** your "after" video is 41 seconds, but the handout asks for about 10. Your video also looks like an earlier version of the game. Mine adds the "Level N" text and the screen shake. For safety, re-record about 10 seconds with this code (a crush, a restart, and gold or red anvils) and replace `after.mp4`.
- **Task 2 text:** the ReadMe's Task 2 description is a copy of Task 1's, so I went by its title, "dynamic difficulty scaling". Tell me if your instructor meant something else.
- **Chat history:** the ReadMe asks for the Chat/LLM page link with the full history. Use this conversation's share link as well as the PDF.

Turn 5 - ChatGPT link

STUDENT
https://chatgpt.com/share/6acb7534-3a34-83e8-8c3c-201fd49492bb

> *[Claude's work between messages]* Fetched the link. Only the page title loaded ('ChatGPT - Anvil Dodge Repair Prompt'); the conversation body is rendered by JavaScript and could not be read. Added the link to Lab-4/chat_link.txt and to the first page of the summary PDF.

CLAUDE
I've added your ChatGPT link to `Lab-4/chat_link.txt` and to the top of `chat_history.pdf`. I couldn't read the chat itself, so I don't know what code it produced.

The commit script already stages `Lab-4`, so `chat_link.txt` goes into the last commit with the after video and the PDF.

The ReadMe's checklist wants the link to the chat you used for the lab. Your link is a ChatGPT chat, but the code in `Lab-4/anvil_dodge/` came from this Claude session. If you submit the ChatGPT link with this code, the two may not match. There are two ways to make that consistent:
- **If you used both tools:** keep both. The PDF says it covers the Claude session and lists the ChatGPT link separately.
- **If the code should come from the ChatGPT chat:** send me that chat's code (copy it from the page) and I'll build the commit snapshots from it instead.

If you push my version, make the share link for this Claude conversation too, since that's the real record behind the code. Your instructor may ask which chat produced which commit.

Turn 6 - Request for the full history

STUDENT
u only generate some history

CLAUDE
You're right, the PDF I made was only a condensed summary. I'll replace it with the complete conversation, covering your messages and the code and ReadMe you pasted, my replies word for word, and the work I did in between.

[This document is that replacement.]

Appendix - Final code produced in this session

main.py (unchanged)

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Anvil Dodge - Pygame Edition")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.update()
        engine.render(screen)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

game/player.py

```
import pygame


class Player:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.width = 44
        self.height = 54
        self.speed = 7

        self.x = (screen_width // 2) - (self.width // 2)
        self.y = screen_height - self.height - 20

    def move_left(self):
        self.x -= self.speed

    def move_right(self):
        self.x += self.speed

    def update(self):
        # Task 1: clamp the player strictly inside the visible window
        self.x = max(0, min(self.x, self.screen_width - self.width))

    @property
    def rect(self):
        return pygame.Rect(int(self.x), int(self.y), self.width, self.height)

    def render(self, surface):
        torso_rect = pygame.Rect(int(self.x) + 6, int(self.y) + 18, 32, 36)
        pygame.draw.rect(surface, (50, 130, 220), torso_rect, border_radius=4)

        pygame.draw.circle(surface, (230, 200, 160), (int(self.x) + 22, int(self.y) + 12), 12)
        helmet_rect = pygame.Rect(int(self.x) + 8, int(self.y) - 2, 28, 12)
        pygame.draw.rect(surface, (180, 180, 190), helmet_rect, border_radius=4)
```

game/anvil.py

```
import random
import pygame


# Task 3: speed tiers -> (top colour, base colour, outline colour)
GREY_TIER = ((120, 120, 130), (80, 80, 90), (200, 200, 210))
WARM_TIER = ((250, 215, 80), (215, 165, 40), (255, 235, 150))   # fast
HOT_TIER = ((255, 110, 60), (205, 45, 45), (255, 170, 140))     # very fast
WARM_SPEED = 6.5
HOT_SPEED = 9.0


class Anvil:
    def __init__(self, screen_width, speed_bonus=0.0):
        self.screen_width = screen_width
        self.width = 40
        self.height = 32
        self.x = random.randint(20, screen_width - self.width - 20)
        self.y = -self.height
        # Task 2: speed_bonus grows with survival time (see GameEngine)
        self.speed = random.uniform(4.5, 7.0) + speed_bonus

    def update(self):
        self.y += self.speed

    def is_off_screen(self, screen_height):
        return self.y > screen_height + 10

    @property
    def rect(self):
        return pygame.Rect(int(self.x), int(self.y), self.width, self.height)

    @property
    def tint(self):
        if self.speed >= HOT_SPEED:
            return HOT_TIER
        if self.speed >= WARM_SPEED:
            return WARM_TIER
        return GREY_TIER

    def render(self, surface):
        top_color, base_color, outline_color = self.tint
        top_rect = pygame.Rect(int(self.x) + 4, int(self.y), self.width - 8, 14)
        pygame.draw.rect(surface, top_color, top_rect, border_radius=2)

        base_rect = pygame.Rect(int(self.x), int(self.y) + 14, self.width, 18)
        pygame.draw.rect(surface, base_color, base_rect, border_radius=3)
        pygame.draw.rect(surface, outline_color, base_rect, width=1, border_radius=3)
```

game/particles.py (new)

```
import random
import pygame

DUST_COLORS = [(190, 170, 140), (160, 145, 120), (215, 195, 160)]


class DustParticle:
    """A short-lived puff of dust kicked up when an anvil hits the ground."""

    def __init__(self, x, y):
        self.x = x + random.uniform(-14, 14)
        self.y = y
        self.vx = random.uniform(-3.0, 3.0)
        self.vy = random.uniform(-3.8, -0.8)
        self.max_life = random.randint(20, 36)
        self.life = self.max_life
        self.radius = random.uniform(2.0, 4.5)
        self.color = random.choice(DUST_COLORS)

    @property
    def alive(self):
        return self.life > 0

    def update(self):
        self.x += self.vx
        self.y += self.vy
        self.vy += 0.18      # gravity
        self.vx *= 0.97      # air drag
        self.life -= 1

    def render(self, surface):
```

```
        alpha = max(0, int(255 * self.life / self.max_life))
        size = int(self.radius * 2) + 2
        puff = pygame.Surface((size, size), pygame.SRCALPHA)
        pygame.draw.circle(puff, (*self.color, alpha), (size // 2, size // 2), int(self.radius))
        surface.blit(puff, (int(self.x) - size // 2, int(self.y) - size // 2))
```

game/game_engine.py

```
import random
import pygame
from game.particles import DustParticle
from game.player import Player
from game.anvil import Anvil


# Task 2: difficulty scaling settings
BASE_SPAWN_DELAY = 700      # ms between anvils at the start
MIN_SPAWN_DELAY = 250       # spawn delay never goes below this
SPAWN_DELAY_DROP_PER_SEC = 12
SPEED_BONUS_PER_SEC = 0.10  # extra anvil speed gained per second survived
MAX_SPEED_BONUS = 6.0
LEVEL_LENGTH_SEC = 10       # a new "level" every 10 seconds

# Task 4: impact FX settings
DUST_PER_IMPACT = 14
SHAKE_FRAMES = 12


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.player = Player(width, height)
        self.anvils = []
        self.particles = []
        self.shake_frames = 0
        self.shake_strength = 0
        self.ground_y = height - 20

        self.spawn_delay = BASE_SPAWN_DELAY
        self.speed_bonus = 0.0
        self.level = 1
        self.last_spawn_time = pygame.time.get_ticks()

        self.start_ticks = pygame.time.get_ticks()
        self.survival_time = 0
        self.game_state = "PLAYING"

        self.font_big = pygame.font.SysFont(None, 52)
        self.font_medium = pygame.font.SysFont(None, 34)
        self.font_small = pygame.font.SysFont(None, 24)

    def handle_event(self, event):
        if self.game_state == "GAME_OVER":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()

    def update(self):
        if self.game_state != "PLAYING":
            return

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move_left()
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move_right()

        self.player.update()

        self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000

        now = pygame.time.get_ticks()
        self.update_difficulty(now)
        if now - self.last_spawn_time >= self.spawn_delay:
            self.anvils.append(Anvil(self.width, self.speed_bonus))
            self.last_spawn_time = now

        player_rect = self.player.rect
        for anvil in self.anvils[:]:
            anvil.update()

            if player_rect.colliderect(anvil.rect):
```

```
            self.game_state = "GAME_OVER"
            break

        # Task 4: anvil smashes into the ground line
        if anvil.rect.bottom >= self.ground_y:
            self.spawn_impact(anvil)
            self.anvils.remove(anvil)
            continue

        if anvil.is_off_screen(self.height):
            self.anvils.remove(anvil)

    for particle in self.particles[:]:
        particle.update()
        if not particle.alive:
            self.particles.remove(particle)

    if self.shake_frames > 0:
        self.shake_frames -= 1

def spawn_impact(self, anvil):
    """Task 4: dust cloud + short screen shake (faster anvils shake harder)."""
    for _ in range(DUST_PER_IMPACT):
        self.particles.append(DustParticle(anvil.rect.centerx, self.ground_y))
    self.shake_frames = SHAKE_FRAMES
    self.shake_strength = min(8, 2 + anvil.speed * 0.4)

def update_difficulty(self, now):
    """Task 2: anvils spawn faster and fall faster the longer you survive."""
    elapsed_sec = (now - self.start_ticks) / 1000
    self.spawn_delay = max(MIN_SPAWN_DELAY,
                           BASE_SPAWN_DELAY - elapsed_sec * SPAWN_DELAY_DROP_PER_SEC)
    self.speed_bonus = min(MAX_SPEED_BONUS, elapsed_sec * SPEED_BONUS_PER_SEC)
    self.level = 1 + int(elapsed_sec // LEVEL_LENGTH_SEC)

def reset(self):
    self.player = Player(self.width, self.height)
    self.anvils.clear()
    self.particles.clear()
    self.shake_frames = 0
    self.start_ticks = pygame.time.get_ticks()
    self.last_spawn_time = pygame.time.get_ticks()
    self.survival_time = 0
    self.spawn_delay = BASE_SPAWN_DELAY
    self.speed_bonus = 0.0
    self.level = 1
    self.game_state = "PLAYING"

def render(self, screen):
    canvas = pygame.Surface((self.width, self.height))
    self.draw_world(canvas)

    # Task 4: screen shake (only while playing, fades out over its duration)
    offset_x = offset_y = 0
    if self.game_state == "PLAYING" and self.shake_frames > 0:
        power = self.shake_strength * self.shake_frames / SHAKE_FRAMES
        offset_x = int(random.uniform(-power, power))
        offset_y = int(random.uniform(-power, power))

    screen.fill((0, 0, 0))
    screen.blit(canvas, (offset_x, offset_y))

def draw_world(self, screen):
    screen.fill((35, 38, 45))

    ground_y = self.ground_y
    pygame.draw.rect(screen, (70, 75, 85), (0, ground_y, self.width, 20))
    pygame.draw.line(screen, (160, 90, 40), (0, ground_y), (self.width, ground_y), 3)

    self.player.render(screen)
    for anvil in self.anvils:
        anvil.render(screen)
    for particle in self.particles:
        particle.render(screen)

    time_surf = self.font_medium.render(f"Survival Time: {self.survival_time}s", True, (240, 240, 240))
    screen.blit(time_surf, (20, 20))

    level_surf = self.font_small.render(f"Level {self.level}", True, (250, 215, 80))
    screen.blit(level_surf, (20, 52))

    inst_surf = self.font_small.render("Use [A/D] or [Arrow Keys] to Dodge", True, (170, 175, 185))
```

```
        screen.blit(inst_surf, (self.width - inst_surf.get_width() - 20, 25))

        if self.game_state == "GAME_OVER":
            overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
            overlay.fill((0, 0, 0, 190))
            screen.blit(overlay, (0, 0))

            over_surf = self.font_big.render("CRUSHED! GAME OVER", True, (235, 65, 65))
            screen.blit(over_surf, (self.width // 2 - over_surf.get_width() // 2, self.height // 2 - 60))

            score_surf = self.font_medium.render(f"You survived: {self.survival_time} seconds", True, (255, 255, 255))
            screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, self.height // 2))

            restart_surf = self.font_small.render("Press [R] to Play Again", True, (200, 200, 200))
            screen.blit(restart_surf, (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 50))
```

Git commits (one per task)

```
Lab 4: add original Anvil Dodge code
Lab 4: add before video (bug behaviour)
Task 1: clamp player inside left/right screen boundaries
Task 2: dynamic difficulty scaling (faster spawns and anvils over time)
Task 3: speed-based anvil warning tints (grey / gold / red)
Task 4: ground impact dust particles and screen shake
Lab 4: add after video and chat history
```